

PROJECT4:
Credit Card Fraud Detection System Using
Logistic / Random Forest
Regression
By
Sindhu Togaru
Doing Internship in
Data Analytics
At
Inno Fortune IT Pvt. Ltd.
Date: 24-03-2026

1. Introduction

In today's digital world, online transactions and card payments have increased rapidly. Along with this growth, financial fraud—especially **credit card fraud**—has also become a major issue. Fraudulent transactions can cause huge financial losses to banks, businesses, and customers.

This project focuses on building a **Machine Learning-based Fraud Detection System** that can automatically identify whether a transaction is **fraudulent** or **legitimate**. The system analyzes transaction data, learns patterns from previous fraud cases, and predicts suspicious transactions with high accuracy.

The project demonstrates how machine learning techniques such as **Logistic Regression** and **Random Forest** can be applied in real-world financial security systems.

2. Objective

The main objective of this project is:

- To develop a **machine learning-based system** that accurately identifies fraudulent credit card transactions.
- To help prevent financial loss by detecting suspicious transactions early.
- To analyze transaction patterns and distinguish between **normal** and **fraudulent** behavior.
- To create a system that can be integrated into a backend application using **Flask/Django**.

3. Problem Statement

Credit card fraud has become one of the most common financial crimes due to the increasing use of online payment systems. Traditional fraud detection methods often fail to detect complex fraud patterns in real time.

Hence, there is a need for an intelligent system that can:

- Analyze large transaction datasets
- Detect unusual behavior
- Minimize false positives
- Improve fraud prevention using machine learning

This project addresses the problem by developing a **Fraud Detection System** using historical credit card transaction data.

4. Scope of the Project

The scope of this project includes:

- Collecting and preprocessing credit card transaction data
- Performing exploratory data analysis (EDA)
- Building classification models to detect fraud
- Evaluating the performance of models
- Integrating the trained model into a backend API
- Optionally creating a simple dashboard or frontend

5. Tools and Technologies Used

The following tools and technologies are used in this project:

Programming Language

- Python

Libraries

- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Machine Learning Models

- Logistic Regression
- Random Forest Classifier

Backend Framework

- Flask / Django (optional)

Frontend (Optional)

- HTML
- CSS

Development Environment

- Jupyter Notebook

6. Dataset Collection

The dataset is collected from **open-source platform such as Kaggle**.

- **Credit Card Fraud Detection Dataset**

Dataset Description

The dataset contains historical credit card transaction records. Each transaction is labelled as:

- **0 → Legitimate Transaction**
- **1 → Fraudulent Transaction**

Common Features in the Dataset

- Time
- Amount
- Transaction-related anonymized features (V1, V2, V3, ..., V28)
- Is fraud (Target Variable)

Target Variable

- **Is fraud**
 - 0 = non-fraud
 - 1 = Fraud

7. Methodology

The project is developed in the following phases:

1. Data Collection
2. Data Preprocessing
3. Exploratory Data Analysis (EDA)
4. Model Development
5. Model Evaluation
6. System Development
7. Testing & Validation
8. Result Analysis
9. Conclusion

8. Data Collection & Preprocessing

8.1 Data Collection

The credit card fraud dataset is collected from open-source platforms such as Kaggle. It contains transaction records including both normal and fraudulent transactions.

8.2 Data Cleaning

The dataset is checked for:

- Missing values
- Duplicate records
- Invalid data entries
- Data type mismatches

8.3 Handling Missing Values

If missing values are found, they are handled using:

- Removal of null records (if very few)
- Mean/median imputation (if necessary)

8.4 Handling Outliers

Outliers are important in fraud detection because fraud transactions may appear unusual. However, extreme noise can affect model performance. Outlier handling methods include:

- Boxplot analysis
- IQR (Interquartile Range) method
- Log transformation (if needed)
- Feature scaling

8.5 Feature Scaling

Since transaction amount may have a different range than other features, scaling is performed using:

- StandardScaler

8.6 Class Imbalance Handling

Fraud datasets are usually highly imbalanced because fraudulent transactions are very rare.

To handle imbalance:

- Random undersampling / oversampling
- SMOTE (Synthetic Minority Over-sampling Technique) (*optional*)
- Class weights in model training

9. Exploratory Data Analysis (EDA)

EDA helps understand the distribution of transactions and identify fraud patterns.

9.1 Fraud vs Non-Fraud Analysis

The number of fraud and non-fraud transactions is compared using:

- Count plots
- Pie charts / bar charts

9.2 Transaction Amount Analysis

The transaction amount is analyzed to compare:

- Average fraud transaction amount
- Average normal transaction amount

9.3 Feature Correlation

A **correlation heatmap** is used to identify relationships between variables.

9.4 Distribution Analysis

Feature distributions are visualized using:

- Histograms
- KDE plots
- Boxplots

9.5 Pattern Identification

EDA helps identify:

- Fraud transactions often have unusual patterns
- Fraud cases are very small in number compared to legitimate transactions
- Certain feature combinations may indicate suspicious behavior

10. Data Visualizations Used

The following charts/graphs are created:

- **Count Plot** – Fraud vs Non-Fraud Transactions
- **Histogram** – Transaction Amount Distribution
- **Boxplot** – Outlier Detection
- **Heatmap** – Feature Correlation
- **Pie Chart / Bar Chart** – Class Imbalance Visualization
- **Scatter Plot** – Relationship between selected features

These visualizations help in understanding the dataset and improving model performance.

11. Model Development

Two machine learning models are used for fraud detection:

11.1 Logistic Regression

Logistic Regression is a simple and effective classification algorithm used for binary classification problems.

Advantages

- Easy to understand
- Fast training
- Works well for binary classification
- Good baseline model

11.2 Random Forest Classifier

Random Forest is an ensemble learning algorithm that combines multiple decision trees.

Advantages

- Handles complex patterns well
- Reduces overfitting
- Works well with imbalanced and nonlinear data
- Usually gives better performance than basic models

12. Model Training Process

The dataset is split into:

- **Training Set** – 80%
- **Testing Set** – 20%

13. Model Evaluation Metrics

Since fraud detection is an imbalanced classification problem, **accuracy alone is not enough**. The following metrics are used:

13.1 Accuracy

Measures overall correctness of predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

13.2 Precision

Measures how many predicted fraud cases are actually fraud.

$$\text{Precision} = \frac{TP}{TP + FP}$$

13.3 Recall

Measures how many actual fraud cases are correctly detected.

$$\text{Recall} = \frac{TP}{TP + FN}$$

13.4 F1-Score

Harmonic mean of Precision and Recall.

$$F1 = 2 \times \left\{ \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \right\}$$

Important Note

In fraud detection, **Recall is very important** because missing a fraud transaction is risky.

14. System Development

The trained model is integrated into a simple backend application.

14.1 Backend Development

Backend is developed using:

- **Flask** (*recommended for simplicity*)
or
- **Django** (*for larger applications*)

14.2 API Integration

A REST API is created to accept transaction data and return prediction results.

Example API Workflow

1. User submits transaction details
2. Backend preprocesses input data
3. ML model predicts fraud/non-fraud
4. API returns result:
 - **Fraud Detected**
 - **Legitimate Transaction**

14.3 Frontend / Dashboard (Optional)

A simple frontend can be built using:

- HTML
- CSS

Dashboard Features

- Input transaction form
- Predict button
- Display result
- Fraud alert message

15. Testing & Validation

After model development, the system is tested with new transaction data.

15.1 Testing Objectives

- Ensure correct fraud classification
- Validate prediction accuracy
- Reduce false positives
- Check API functionality
- Confirm model stability

15.2 Validation Process

- Use unseen test data
- Check confusion matrix
- Evaluate Precision, Recall, F1-score
- Compare Logistic Regression vs Random Forest
- Identify incorrect predictions

15.3 Bug Fixing & Improvements

- Fix data preprocessing errors
- Handle unexpected input values
- Improve model threshold
- Reduce unnecessary fraud alerts

16. Expected Results

The project aims to achieve:

- High fraud detection accuracy
- Effective identification of fraudulent transactions
- Improved financial security
- Reduction in financial losses
- Better performance using machine learning than traditional rule-based systems

17. Advantages of the System

- Fast and automated fraud detection
- Reduces manual monitoring effort
- Helps prevent financial loss
- Can detect hidden fraud patterns
- Scalable for real-world applications
- Can be integrated into banking systems

18. Conclusion

The **Fraud Detection System using Machine Learning** successfully demonstrates how intelligent systems can be used to improve financial security by identifying suspicious transactions.

By using **Credit Card Fraud Detection**, the project shows how machine learning algorithms such as **Logistic Regression** and **Random Forest** can analyze transaction data and classify fraud cases effectively.

This project helps in:

- Detecting fraudulent transactions accurately
- Reducing financial losses
- Improving transaction security
- Supporting automated fraud prevention systems

Overall, the project proves that machine learning is a powerful and practical solution for modern fraud detection challenges in the financial domain.